BTL1213 Operating Systems / BIT1213 Operating Systems / BCL1213 Operating Systems

SEGi University

Chapter 02 Process Management



BACHELOR OF INFORMATION TECHNOLOGY (HONS)

BACHELOR OF COMPUTER SCIENCE (HONS)

By: RUFFINUS RUSELL

Learning Objectives:

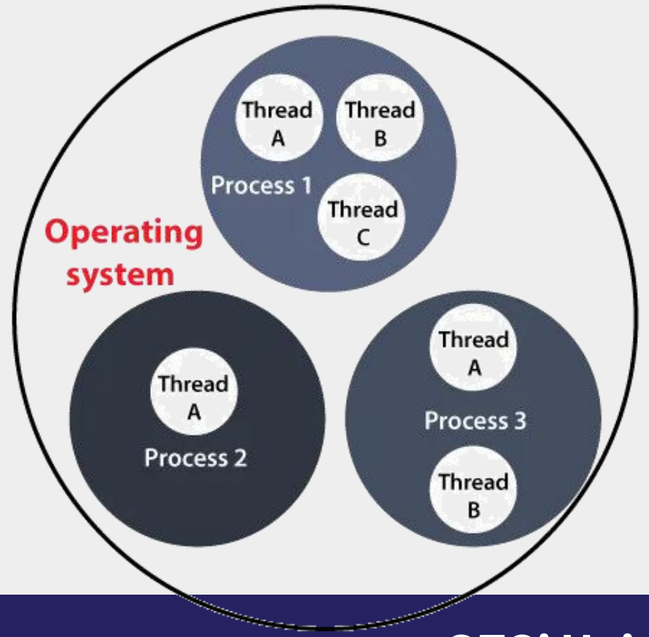
By the end of this lesson, students will be able to:

- Explain the concept of a process and threads and its role.
- Describe the different states in the process lifecycle.
- Differentiate between processes and threads.
- Explain the concept of concurrency and it's comparison with parallelism.

OVERVIEW

Topic for today's main discussion:

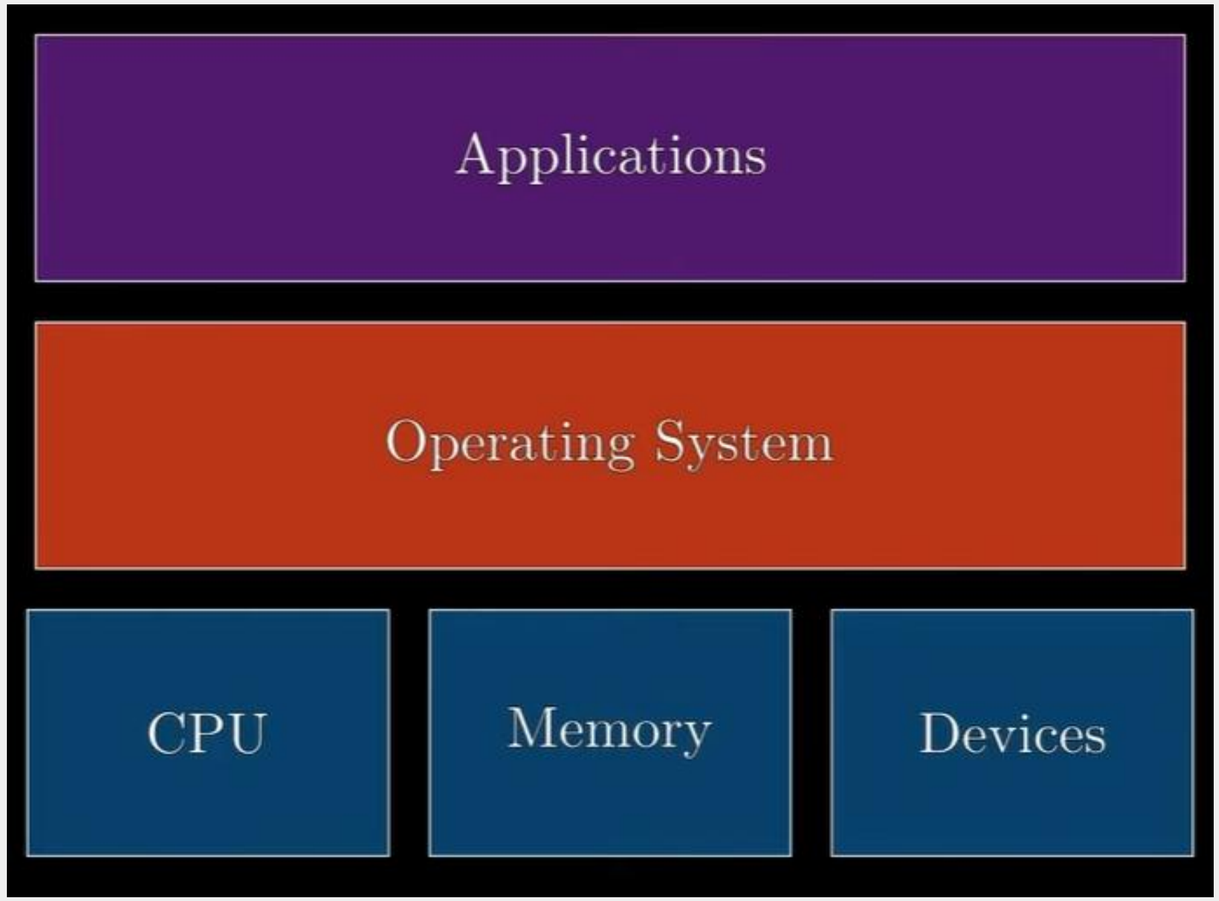
- What is Process Management?
- What is Threads?
- Process vs Thread
- What is Concurrency?
- Concurrency vs Parallelism



Introduction

- Modern operating systems are designed to handle multiple tasks efficiently while maintaining good performance and responsiveness.
- These are mainly achieved through processes and threads.
- Process and threads are fundamental concepts in any multitasking operating system.

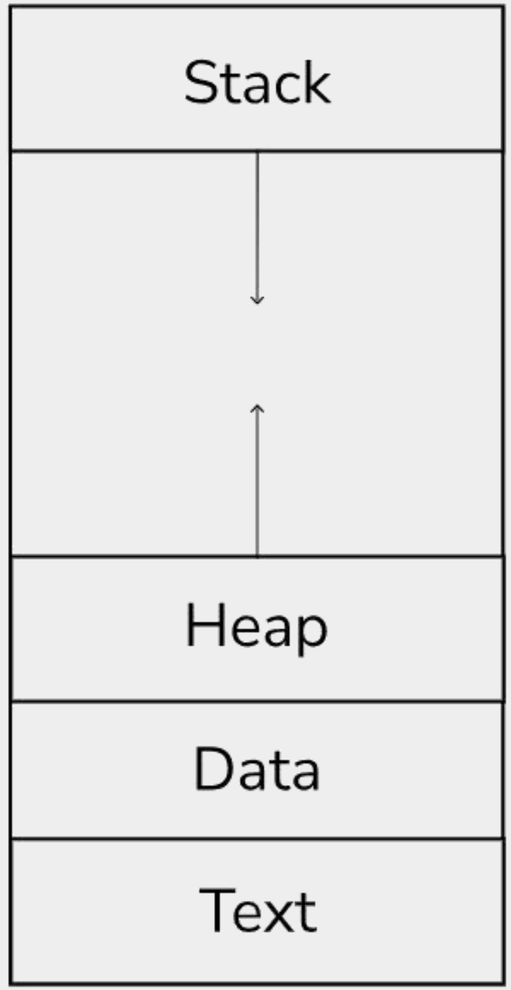
01 What is Processess?



01 What is Processess?

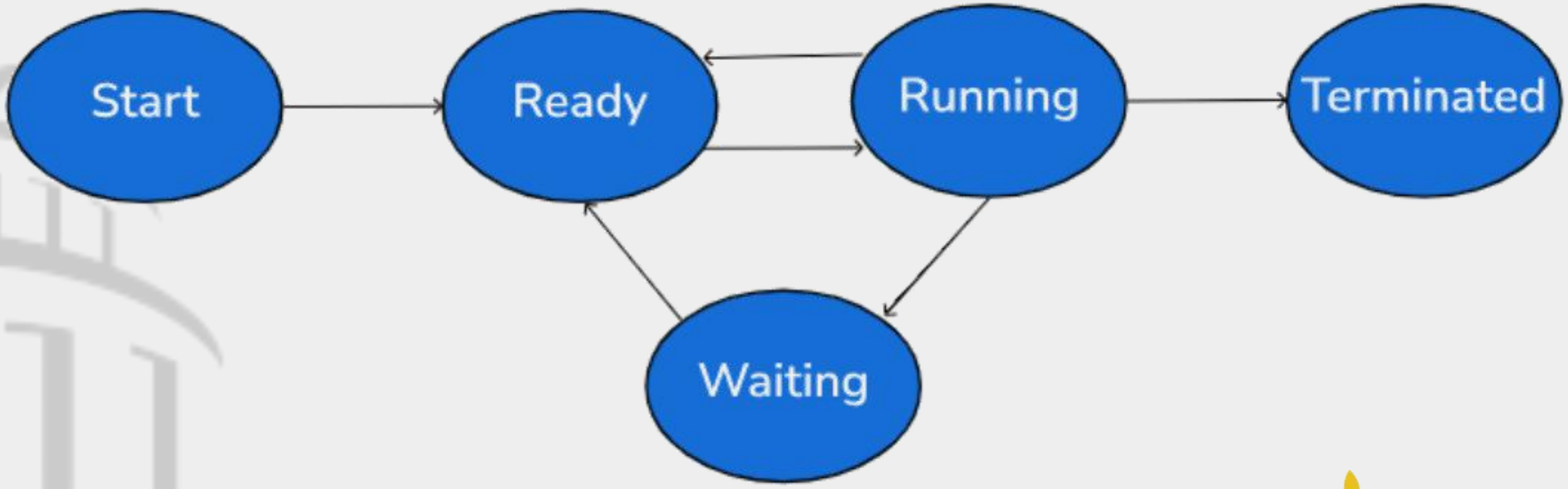
- Process is a passive program loaded into memory and run by the CPU.
- It has its own dedicated and isolated memory address space (code, data, heap).
- Each process has a unique Process ID (PID) for identification.
- Processes communicate with each other using Inter-Process Communication (IPC) methods.

- A process consists of the following resources :
  - Text (machine code of the compiled program).
  - Data (global, static, constant, and uninitialized variables).
  - Heap (dynamic memory allocation).
  - Stack (temporary data such as function paremeters and returns addresses)



Process State

- As a process executes, it moves through different process states managed by the operating system.
- These called Process States and Lifecycle:

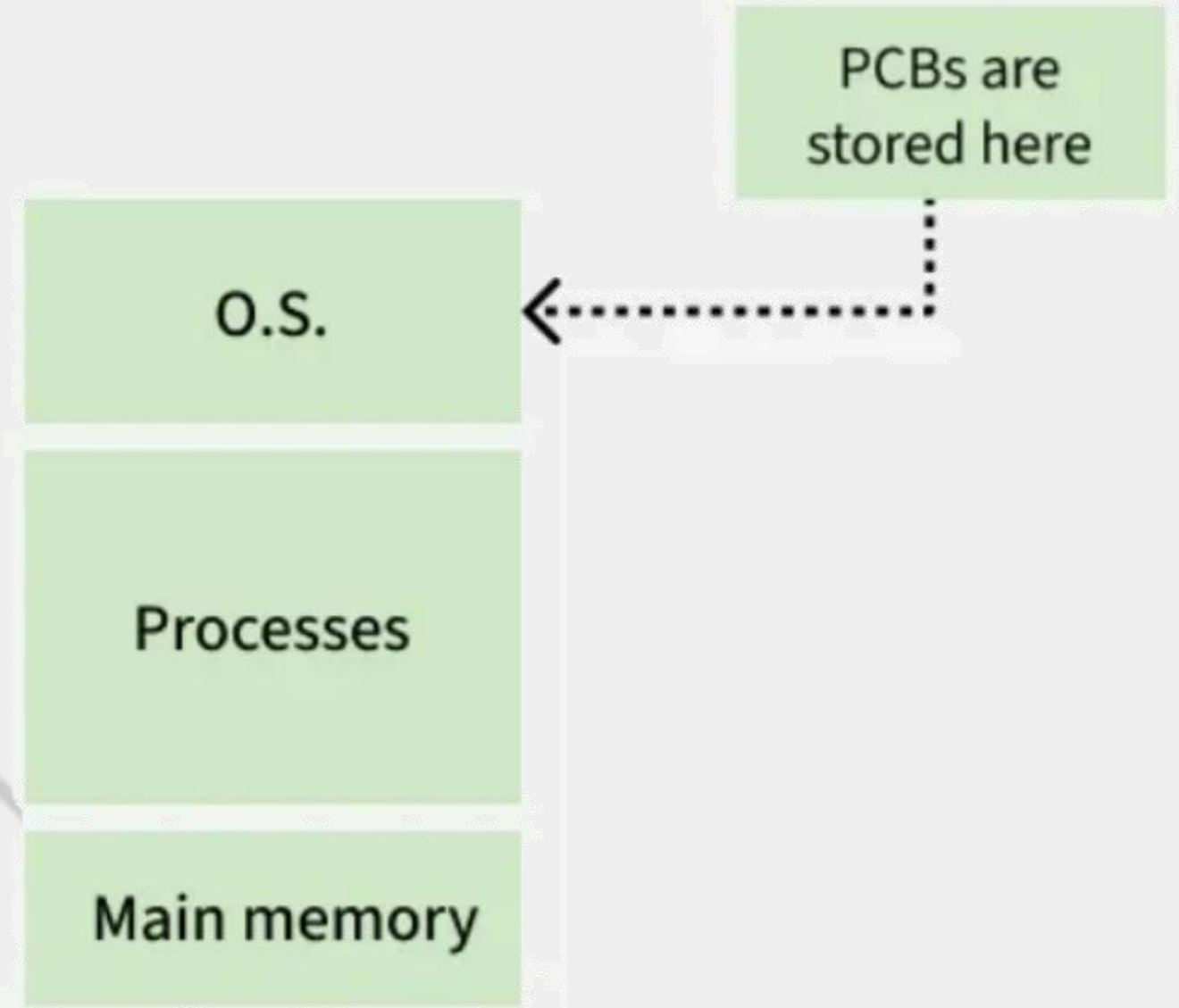


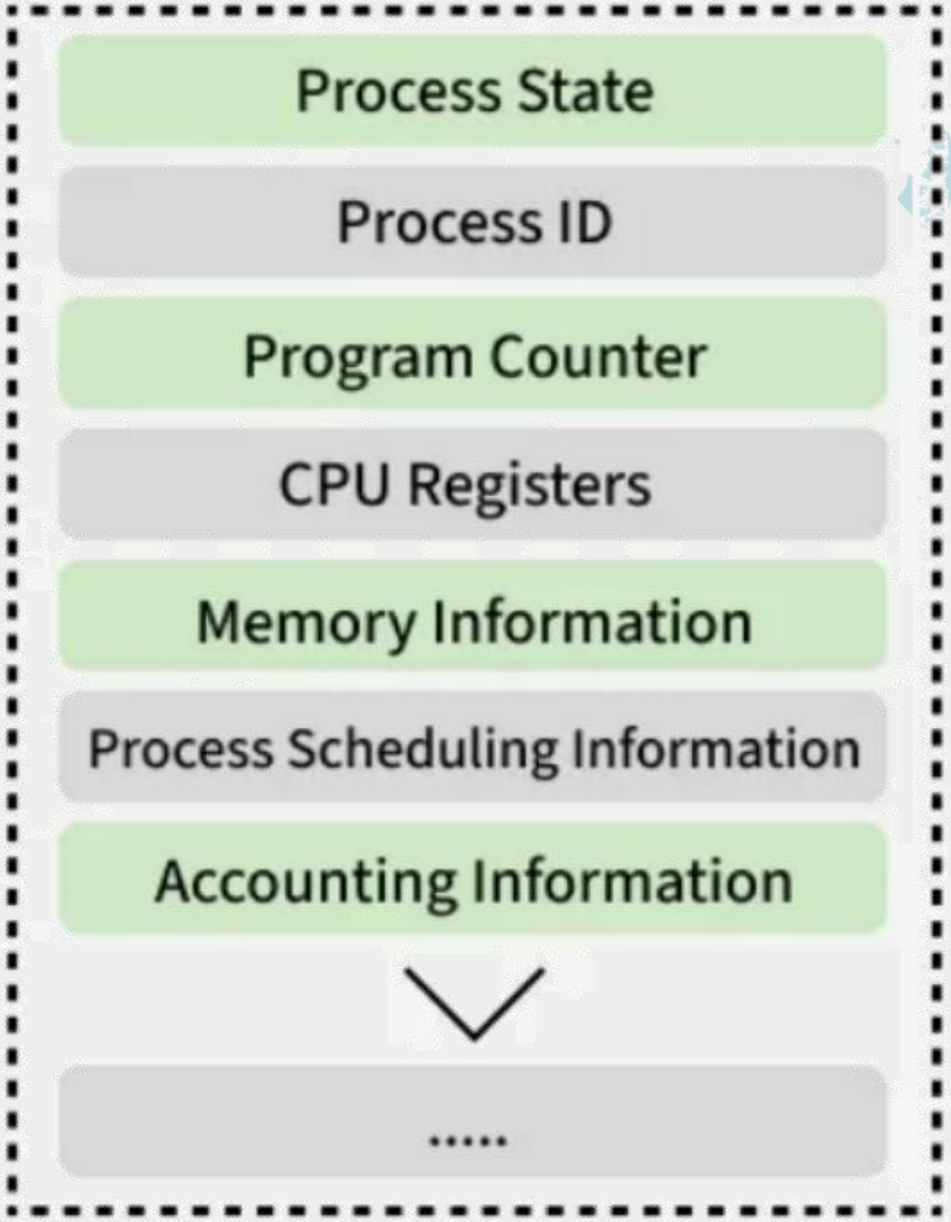
- A process goes through different states in its entire lifecycle. A process goes through the following states:
  - **Start:** This is the initial state. A process is in this state on its creation.
  - **Ready**: The process is assigned 'Ready' state when it is ready to be processed. The process is moved from the 'Start' state to the 'Ready' state after it is moved into memory.

- **Running:** The process is moved to the 'Running' state when it is assigned to a processor and its processing starts. At a single point in time, a single process will be in the 'Running' state on a single processor.
- **Waiting:** The process is moved to the 'Waiting' state if the process is waiting for any resource.
- **Terminated:** The process is moved to the 'Terminated' state once its execution is completed or it is terminated by the OS.

Process Control Block (PCB)

- While the process can have the following states, PCB (Process Control Block) holds the context of process.
- A PCB is a data structure used by the operating system to keep track of process information and manage execution.
- It helps the OS monitor and control process execution.


PCBs are stored here
O.S.
Processes
Main memory


Process State
Process ID
Program Counter
CPU Registers
Memory Information
Process Scheduling Information
Accounting Information
.....

Process Schdulers in Operating System (in Chapter 4):

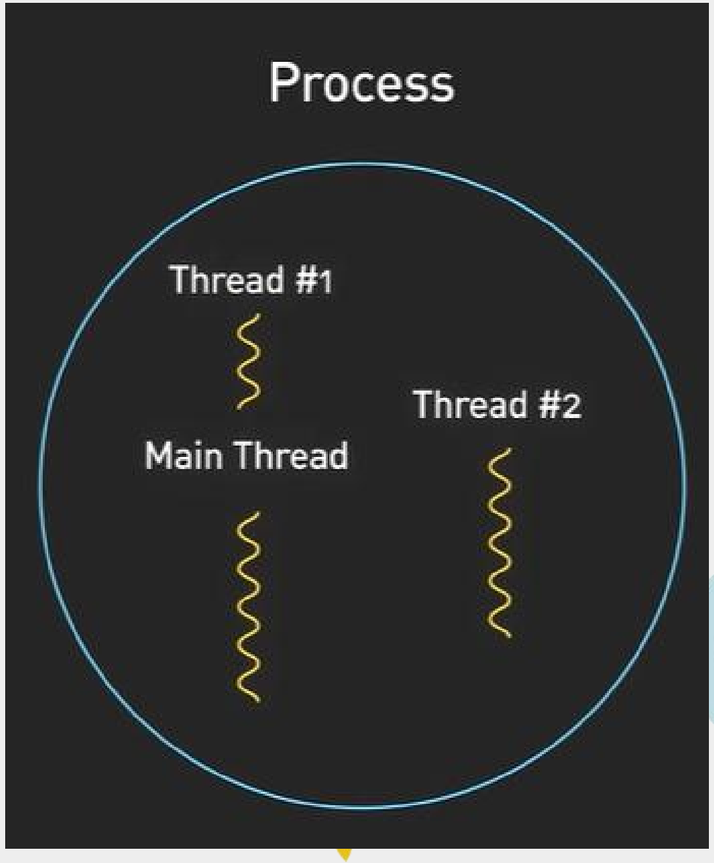
- Process Schedulers are fundamental components of os for deciding the order in which processes are executed by the CPU.
- But why process schdulers?



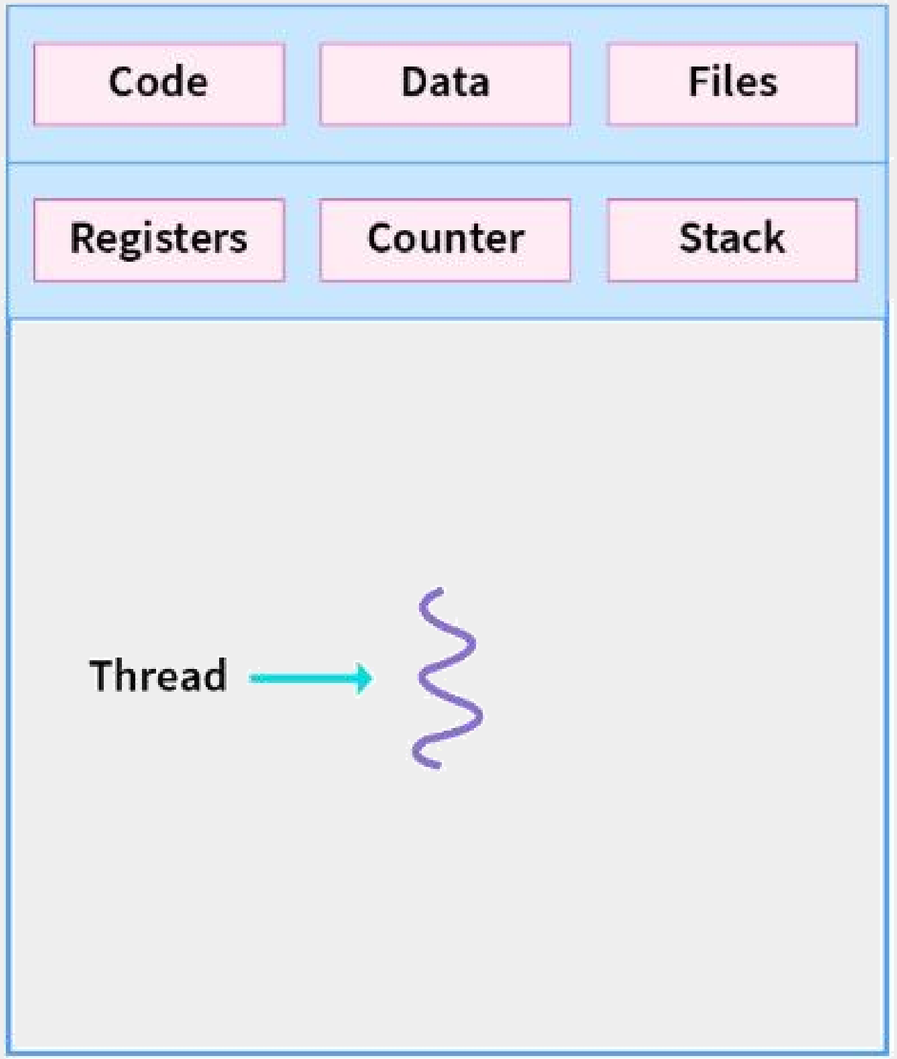
Advantages	Disadvantages
Isolation – Prevents data corruption between processes.	**High Memory Use** – Each process needs separate memory.
Stability – One process crash doesn't affect others.	**Slow IPC** – Inter-process communication is complex.
Security – Separate memory enhances protection.	**High Overhead** – More CPU/memory needed for management.
Concurrency – Multiple processes run simultaneously.	**Slow Context Switching** – Switching between processes is costly.
Efficient Scheduling – OS manages tasks effectively.	**Complex Scheduling** – Requires advanced algorithms.

02 What is Threads?

- Thread is a smallest unit of execution within a process.
- A proces has at least one thread.
- It enables a program to perform multiple tasks concurrently while sharing the same memory and resources.
- Each thread has its own program counter, stack, and set of registers.



- Threads improve application performance and responsiveness in multitasking environments.
- Threads are also termed **lightweight processes** as they share common resources.
- **Eg: While playing a movie on a device the audio and video are controlled by different threads in the background.**



Single-threaded process

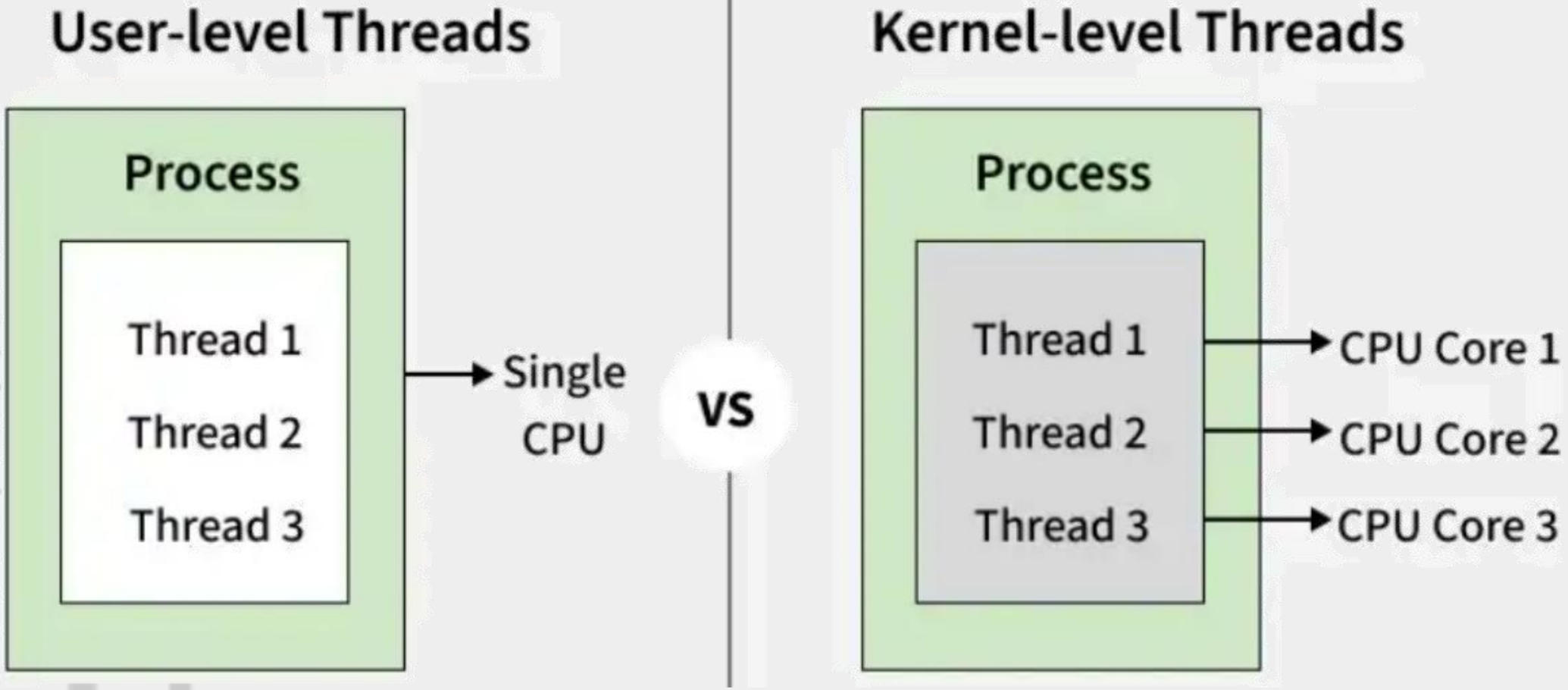
- Each thread has its own Thread ID (TID) for identification.
- A thread also moves through states such as new, runnable, running, waiting, and terminated.
- Threads within the same process share memory and resources, enabling faster communication.
- Context switching can occur between threads to allow multiple tasks to execute efficiently.

- Each thread has its own :
  a) Program Counter
  b) Register Set
  c) Stack Space
- Threads share code, data, and operating system resources within the same process.

Benefits

- Threads provide multiple benefits and improve the overall performance of the system. Some of the reasons are:
  - Since threads use the same data and code, the operational cost between threads is low.
  - Creating and terminating a thread is faster compared to creating or terminating a process.
  - Context switching is faster in threads compared to processes.

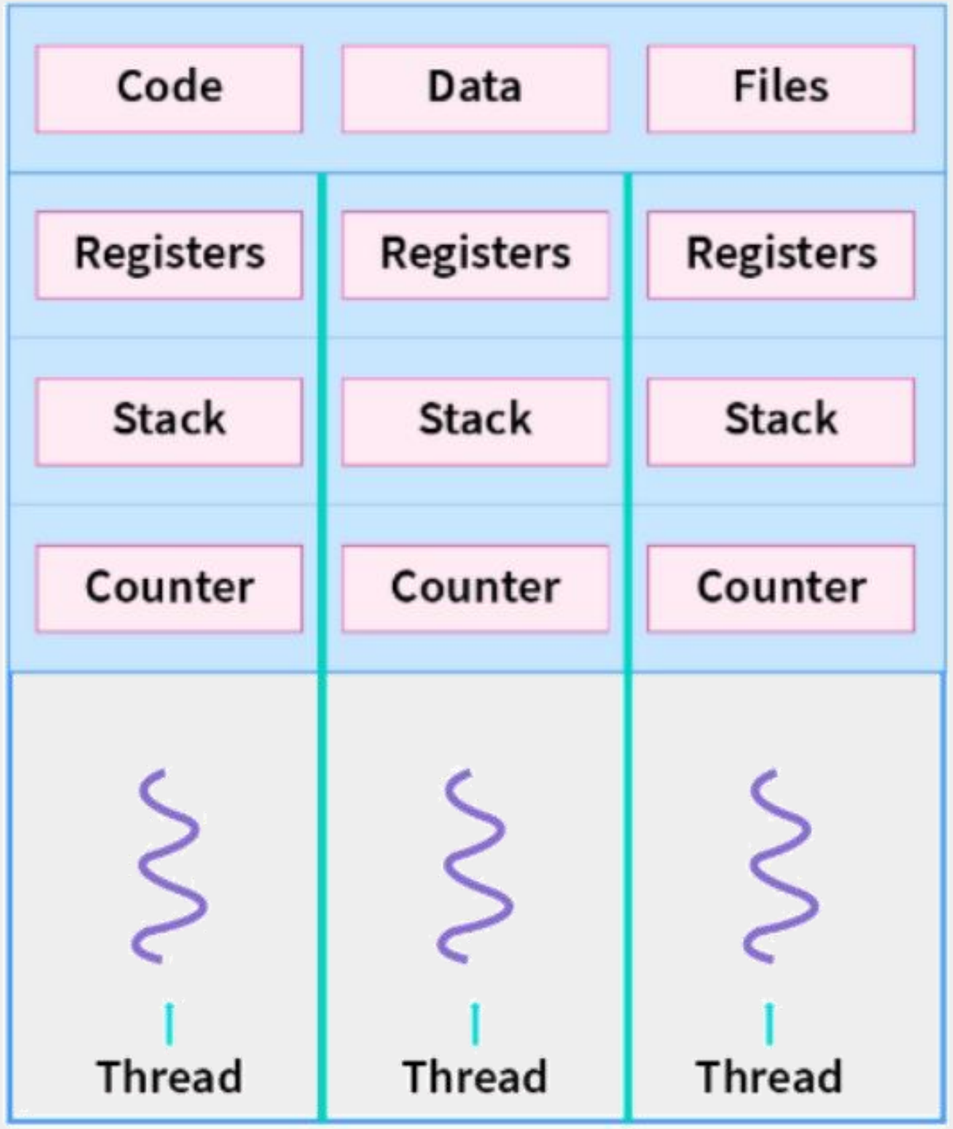
Types of Threads



USER LEVEL THREAD	KERNEL LEVEL THREAD
User thread are implemented by users.	kernel threads are implemented by OS.
OS doesn't recognized user level threads.	Kernel threads are recognized by OS.
Implementation of User threads is easy.	Implementation of Kernel thread is complicated.
Context switch time is less.	Context switch time is more.
Context switch requires no hardware support.	Hardware support is needed.
If one user level thread perform blocking operation then entire process will be blocked.	If one kernel thread perform blocking operation then another thread can continue execution.
Example : Java thread, POSIX threads.	Example : Window Solaris.

Why Multithreading?

- In Multithreading, the idea is to divide a single process into multiple threads instead of creating a whole new process.
- Multhreading offers resource sharing, responsiveness and more economical usage.

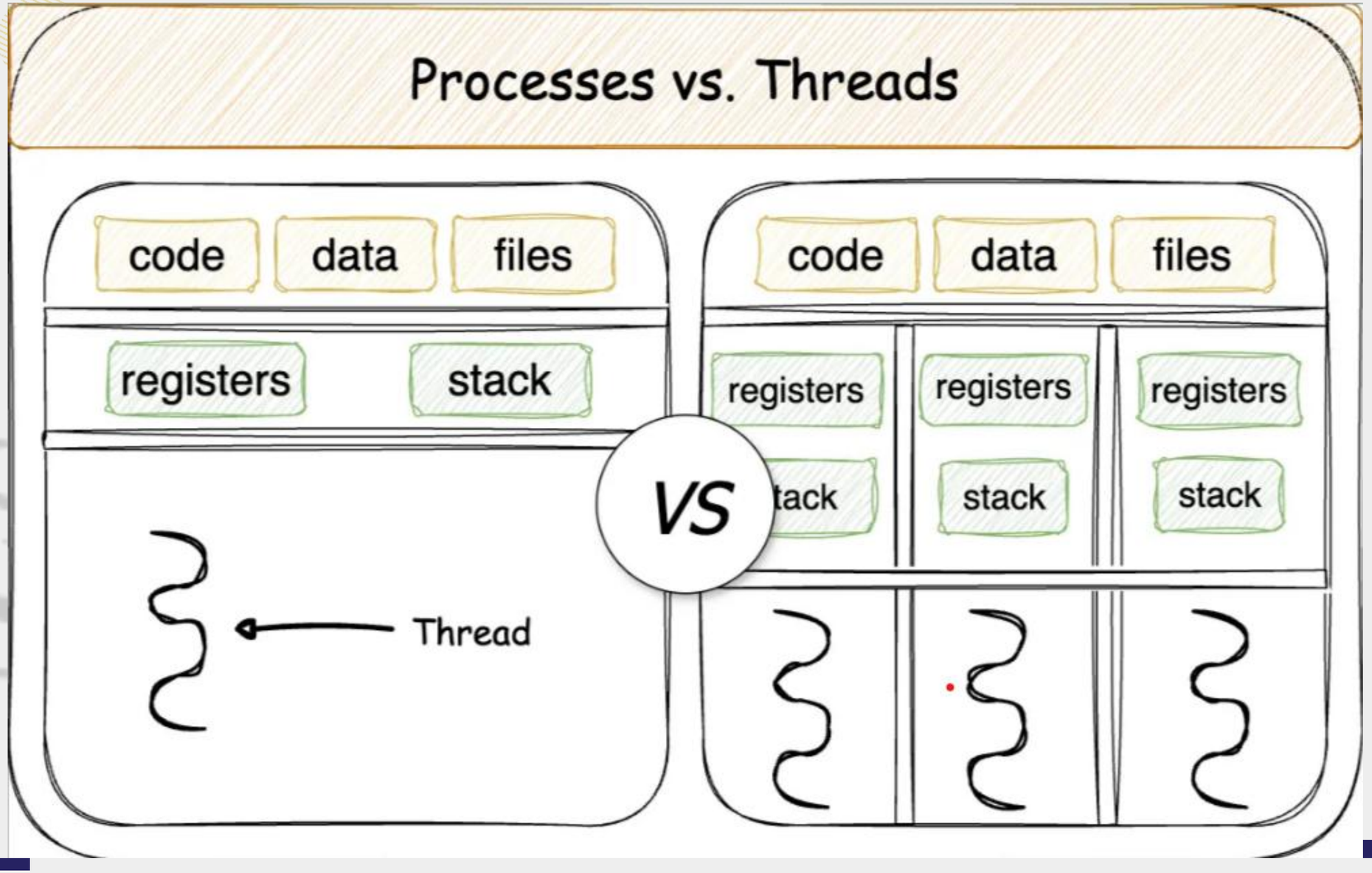


Multithreaded process

Advantages	Disadvantages
Faster Execution – Threads share memory, reducing overhead.	**Security Risk** – One thread can affect others in the same process.
Efficient CPU Usage – Allows multitasking and better performance.	**Complex Synchronization** – Needs careful handling to avoid race conditions.
Quick Context Switching – Faster than process switching.	**No Isolation** – A thread crash can impact the whole process.
Simplified Communication – Threads share data easily.	**Deadlocks** – Poor synchronization can cause deadlocks.
Less Resource Usage – Uses less memory compared to processes.	**Difficult Debugging** – Harder to find and fix thread-related issues.

03 Process vs Thread

- The primary difference is that different processes cannot share the same memory space (code, variables, etc) whereas different threads in the same process share the same memory space.
- Threads are lightweight whereas Processes are heavyweight.
- Threads switching does not need to interact with the OS whereas process switching requires interaction with OS.

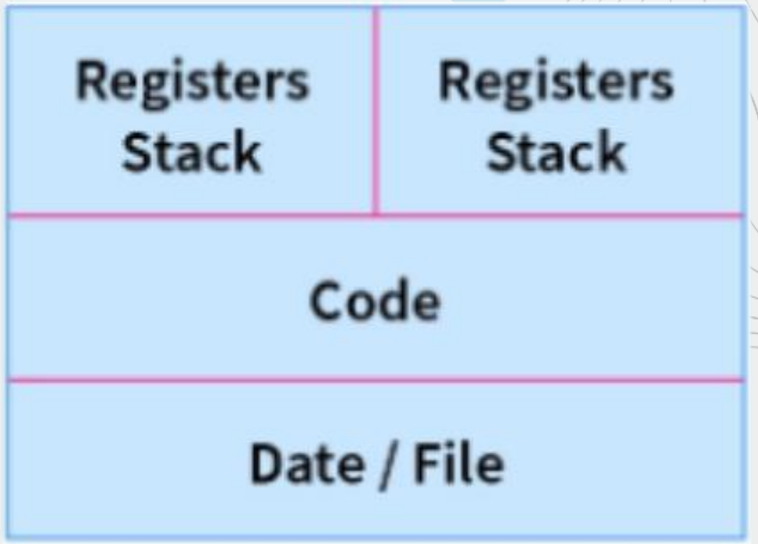

Processes vs. Threads
code
data
files
registers
stack
VS
Thread
code
data
files
registers
registers
registers
stack
stack

PROCESS



Two Different Process

THREAD



Two Threads of a single process

Process	Thread
Processes use more resources and hence they are termed as heavyweight processes.	Threads share resources and hence they are termed as lightweight processes.
Creation and termination times of processes are slower.	Creation and termination times of threads are faster compared to processes.
Processes have their own code and data/file.	Threads share code and data/file within a process.
Communication between processes is slower.	Communication between threads is faster.
Context Switching in processes is slower.	Context switching in threads is faster.
Processes are independent of each other.	Threads, on the other hand, are interdependent. (i.e they can read, write or change another thread's data)
Eg: Opening two different browsers.	Eg: Opening two tabs in the same browser.

04 Concurrency

- Concurrency in operating systems refers to the ability of an OS to manage and execute multiple tasks or processes simultaneously.
- It allows multiple tasks to overlap in execution, giving the appearance of parallelism even on single-core processors.
- Concurrency is achieved through various techniques such as multitasking, multithreading, and multiprocessing.

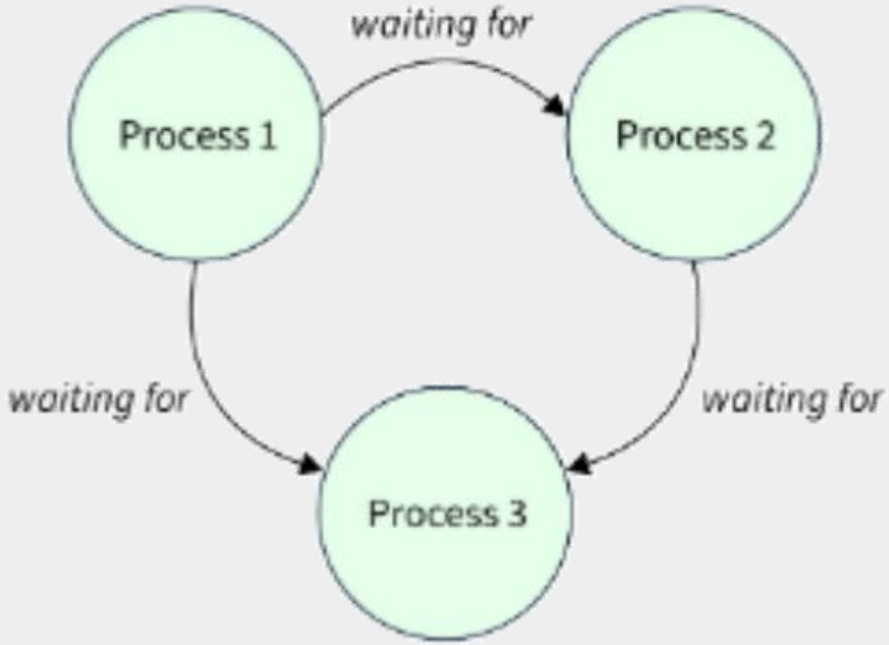

Concurrency
Task 1
Execution
1/0 wait
Execution
Task 2
Execution
1/0 wait
Execution
Task 3
Execution
1/0 wait
Execution

- Why Concurrent occurs?
    a) Resource utilization
    b) Responsiveness
    c) Throughput
    d) Real-Time Processing

- Principles of Concurrency in Operating System
  - a) Process Isolation : maintain system stability
  - b) Synchronization : shared resources and ensure data consistency
  - c) Deadlock Avoidance : Help to halt entire system
  - d) Fairness : CPU time fairly distributed

Advantages	Disadvantages
Better CPU Utilization – Maximizes resource use.	**Complex Synchronization** – Requires careful handling of shared resources.
Faster Execution – Multiple tasks run in parallel.	**Deadlocks** – Poor management can cause processes to wait indefinitely.
Responsive System – Improves user experience.	**Race Conditions** – Multiple processes accessing shared data can cause errors.
Supports Multitasking – Multiple applications can run simultaneously.	**Higher Overhead** – Requires more CPU and memory management.

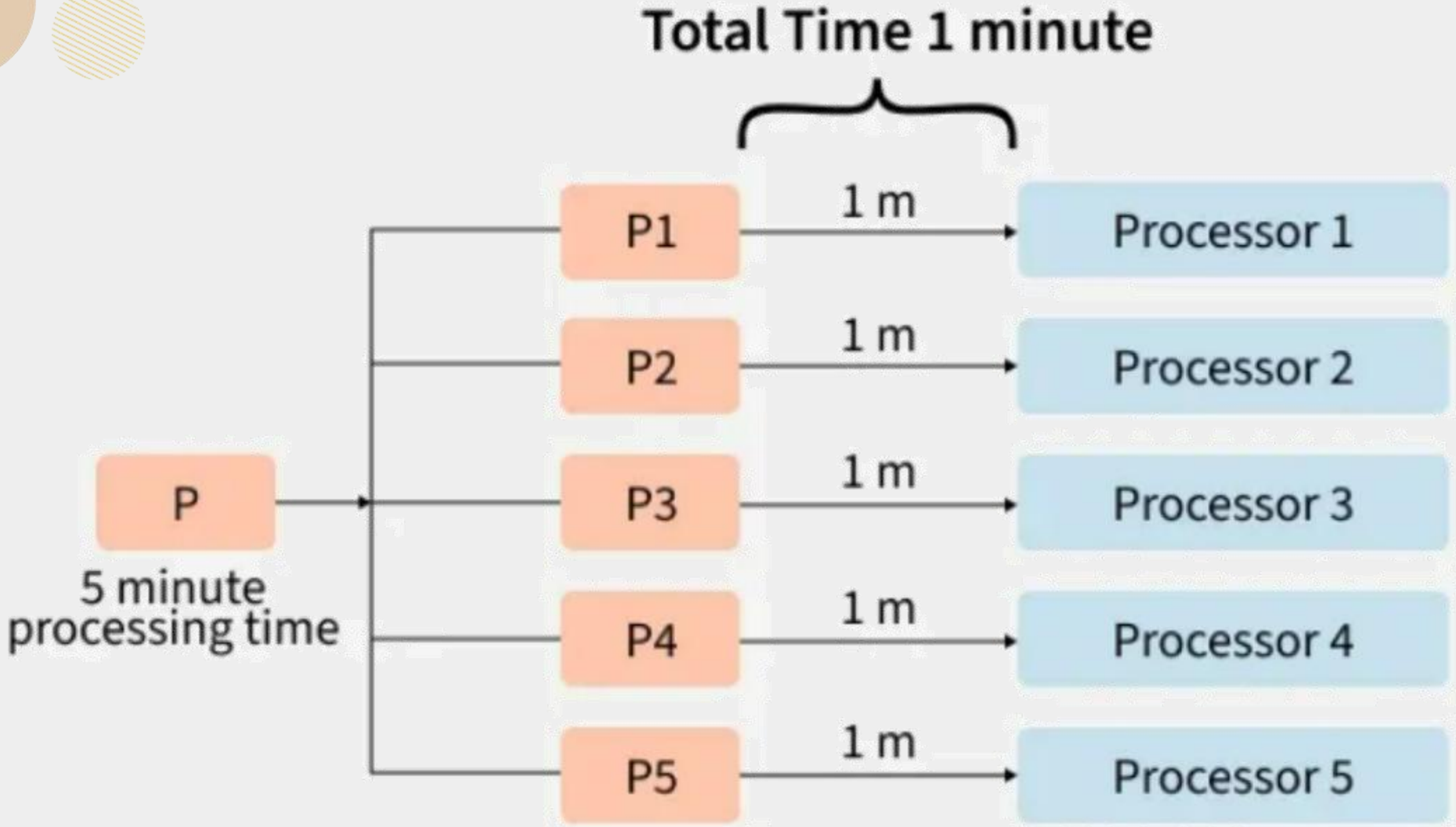




*Example of complexity in Concurrency

Parallelism

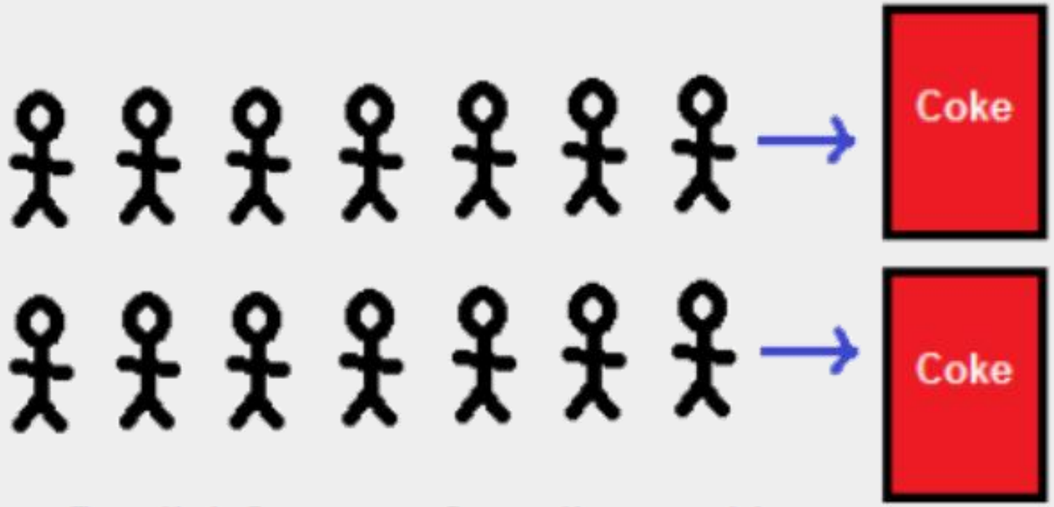
- Parallelism refers to executing multiple tasks simultaneously using multiple processing units.
- It improves system throughput and computational speed by dividing work across processors.
    - Tasks are split into smaller subtasks that run in parallel on separate cores or processors.
    - Commonly used in data processing and high-performance applications.


Total Time 1 minute
P
5 minute
processing time
P1
P2
P3
P4
P5
1 m
1 m
1 m
1 m
1 m
Processor 1
Processor 2
Processor 3
Processor 4
Processor 5

Concurrency vs Parallelism



Concurrent: 2 queues, 1 vending machine



Parallel: 2 queues, 2 vending machines

Concurrency	Parallelism
Concurrency is the task of running and managing the multiple computations at the same time.	While parallelism is the task of running multiple computations simultaneously.
Concurrency is achieved through the interleaving operation of processes on the central processing unit(CPU) or in other words by the context switching.	While it is achieved by through multiple central processing units(CPUs).
Concurrency can be done by using a single processing unit.	While this can't be done by using a single processing unit. it needs multiple processing units.
Concurrency increases the amount of work finished at a time.	While it improves the throughput and computational speed of the system.
Concurrency deals with a lot of things simultaneously.	While it does a lot of things simultaneously.
Concurrency is the non-deterministic control flow approach.	While it is deterministic control flow approach.
In concurrency debugging is very hard.	While in this debugging is also hard but simple than concurrency.

- Concurrency: Like a single cashier serving multiple customers by switching between them very quickly (switching between tasks)
- Parallelism: Like multiple cashiers serving multiple customers at the same time (tasks running truly at the same time)



Real World Scenario

- For instance, how should Chrome handle the fact that many different tasks often happen at the same time when using a browser? Every browser window (or tab) may communicate with several servers on the internet to download audio, video, text, and other resources. In addition, many users have 10 to 20 browser tabs (or more…) open most of the time, and each of these tabs may perform multiple tasks.

SOLUTION:

- They chose to run each browser window in Chrome as a separate process rather than a thread or many threads.
- Running each window as a process protects the overall application from bugs and glitches.
- Isolating a JavaScript program in a process prevents it from using too much CPU time and memory.

Summary

- Process and Thread are related to each other and quite similar as these are the independent sequence of execution. **The difference between a process and a thread is that a process takes place in different memory spaces, whereas a thread executes in the same memory space.**
- Meanwhile, concurrency may be supported by multi-threading or multi-processing whereby more than one process or threads are executed simultaneously or in an interleaved fashion.

THANK YOU



